人工智能数学原理与算法

实验 05

宠物主体分割

从卷积编码器到小型 U-Net

学生讲义与实验手册

主讲　钟志诚

整理　曾有成

单位　中国科学技术大学
人工智能与数据科学学院

本次实验要完成的作品

输入一张宠物照片，输出主体区域。自己搭建卷积编码器和解码器，在本地 NVIDIA 显卡或学校 107 平台训练和评估。进阶任务通过跳跃连接探索细节恢复。

2026 年秋季　3 学时（135 分钟），完整训练与对比可课后完成。
框架：PyTorch　数据：Oxford-IIIT Pet　训练与评估：本地 NVIDIA 显卡或 107 平台。

目录

1 代码结构与数据准备 1

2 实验目标与课堂安排 3

3 T1：数据、标注与划分 4

4 T2：卷积、激活与池化怎样执行 5

5 T3：自己搭建编码器 7

6 T4：自己搭建解码器 8

7 损失、梯度与分割指标 10

8 T5：本地 GPU 与 107 平台两种运行方式 12

9 进阶选做：自行实现小型 U-Net 14

10 联系 ResNet：跳跃连接怎样组合特征 15

11 改进提示：归一化与泛化 16

12 T6：对比、评分与最终提交 17

13 复习与符号索引 18

14 参考资料与学习接口 19

阅读顺序

先按第 1 节准备代码和数据，再完成第 2–8 节的基础任务；第 9–11 节为进阶与改进提示。各节依次说明输入、计算过程、输出与实验要求。源代码中的 TODO 表示待补全位置，Hint 表示提示。

1 代码结构与数据准备

1.1 从哪些文件开始

解压学生包后，code/ 存放实验代码，slides/ 存放课堂课件；report-template.md 是报告模板，提交材料的目录与打包命令见《提交与打包说明》。下表中的路径均相对于code/。

文件或目录	作用与完成要求
operator_exercises.py	学生补全小矩阵卷积、激活、池化三个函数
student_encoder.py	学生补全基础版与改进版编码器
student_decoder.py	学生补全解码器的卷积块、上采样与输出头
models.py	已提供编解码器连接与模型选择入口
data.py、prepare_data.py	读取与变换数据；提供划分生成函数，数据包已含固定划分
check_data.py	检测数据文件、固定划分、图片与掩码
preview_data.py	生成训练样本及掩码预览图
checks.py	检查基础运算、各层尺寸、梯度与参数更新
train.py	训练、逐轮验证、保存最优模型与训练记录
evaluate.py	读取选定权重，计算测试指标并保存预测图
metrics.py、visuals.py	已提供损失、指标、训练曲线与预测图绘制
run_local.py、scripts/	本地运行入口与 107 平台 Slurm 脚本，均保存日志

1.2 数据怎样经过这些模块

train.py 从data.py 读取图片与掩码，通过models.py 建立网络。图片依次经过学生编码器和解码器，得到逐像素原始得分；metrics.py 据此计算损失和指标。每轮结束后记录验证结果，并由visuals.py 绘制曲线和预测图。evaluate.py 读取选定模型，在测试集上生成最终结果。

基础任务补全三个学生文件，分别训练基础版与改进版。小型 U-Net 为独立选做，学生依据后文结构示意自行实现。每组训练产出配置、逐轮指标、曲线、模型权重和代码快照。

1.3 进入代码目录与准备环境

参考环境为 Python 3.11、PyTorch 2.7.1、Torchvision 0.22.1。在解压后的实验目录打开终端，进入code/ 并安装依赖；后续命令均在此目录执行：

```
cd code
python -m pip install -r requirements.txt
```

已配置相同依赖的环境可直接使用。本地 CPU（Central Processing Unit，中央处理器）用于基础运算和编解码器检查。完整训练使用 GPU（Graphics Processing Unit，图形处理器）：有本地 NVIDIA GPU 的同学按第 8.1 节运行；其余同学完成本地 CPU 检查后，转到第 8.2 节使用 107 平台 Slurm。两条路径共用代码、数据划分和评分标准。

1.4 从科大网盘获取数据包并解压

从课程提供的科大网盘下载`lab05-pet-data.zip`。数据包已包含宠物照片、三值掩码和本实验的固定划分。

Windows：右键单击压缩包，选择“全部解压”，将解压得到的`data`文件夹放入学生包的`code/`目录。最终划分文件的路径为`code/data/splits.json`。

Linux、macOS 或 107 平台：将压缩包放入`code/`，在该目录执行：

```
unzip lab05-pet-data.zip
```

解压完成后，各系统均在`code/`目录运行数据检测和预览：

```
python check_data.py --data-dir data
python preview_data.py --data-dir data
```

固定划分`splits.json`已随数据包提供，直接用于训练与评估。

1.5 数据集检测脚本检查什么

`check_data.py`在 CPU 上读取固定划分及其对应的全部数据，检查以下内容：

- 目录中具有`splits.json`与官方训练、测试名单；划分内容与课程固定版本一致。
- 训练 2941 张、验证 739 张、测试 3669 张对应的图片和掩码均存在且可读取。
- 每张原图与其掩码的空间尺寸相同，掩码像素值属于 1、2、3。

检测结束后显示“数据集检测通过：7349 对图片与掩码”。随后运行的`preview_data.py`将训练样本及掩码预览保存为`outputs/data_preview.png`，用于观察数据内容。

1.6 准备后的目录与平台路径

```
code/
  data/
    splits.json
    oxford-iiit-pet/
      images/                 # *.jpg
      annotations/
        trainval.txt
        test.txt
        trimaps/              # *.png
  outputs/
    data_preview.png
```

训练读取解压后的图片、掩码与`splits.json`，数据压缩包可另存。选择 107 平台路径时，将完整`code/`及其中的`data/`上传到平台；若数据另放在共享目录，`LAB05_DATA_DIR`填写包含`splits.json`的那一级目录。使用上述目录布局时，在平台`code/`中设置：

```
export LAB05_DATA_DIR="$PWD/data"
```

`LAB05_PYTHON`用于指定平台 Python 解释器，Slurm 作业提交与日志读取见第 8.2 节。

2 实验目标与课堂安排

2.1 从分类走向逐像素预测

正课已经介绍 CNN（Convolutional Neural Network，卷积神经网络）的局部连接、权重共享、池化与残差连接。本实验把这些结论用于一个具体任务：从照片中找出宠物主体。分类只给整张图一个类别；**语义分割**要给每个像素一个类别。本实验采用二类语义分割：宠物像素属于前景，其余有效像素属于背景。

完成实验后，应能：解释卷积、激活与池化的作用；填写各层尺寸和参数量；自己搭建编码器；在固定划分上完成模型选择；解释分割误差；比较 U-Net 拼接与 ResNet（Residual Network，残差网络）的相加连接。

2.2 实验任务与分工

部分	学生完成	已提供
基础运算	小矩阵卷积、激活、池化	可运行示例与自检
网络	基础和改进编码器、基础解码器	网络连接、训练程序
数据与评价	观察样本、解释指标	数据划分、掩码转换、指标实现
运行与分析	选择运行路径、对比、报告	本地入口、Slurm 脚本和报告模板
进阶（选做）	自行实现小型 U-Net 并接入训练	结构示意和维度提示

2.3 135 分钟课堂流程

时间	任务	教师活动	学生活动与可观察产出
0–15	T1	解释任务和三值标注	查看样本，说出忽略区域
15–35	T2	演示一次滑窗计算	完成手算与三个小函数
35–55	T3 前半	解释通道与尺寸	搭建阶段 1，核对输出
55–70	T3 后半	指导阶段连接	完成三阶段尺寸表
70–90	T4	解释上采样和输出头	完成解码器与一次更新
90–115	T5	演示本地与 Slurm 入口	启动训练，读取记录
115–135	T6/进阶	讲解结果分析与拼接	完成对比计划和退出卡

课堂时间合计 135 分钟。环境与数据在课前准备；课堂完成基础网络和训练提交，完整训练与对照分析可在课后完成。

3 T1：数据、标注与划分

3.1 同一张图的三个角色

Oxford-IIIT Pet 数据由牛津大学等团队发布，包含猫狗照片与像素级三值标注（trimap）。**掩码**是与图片空间位置对应的标签数组。输入使用 RGB（Red, Green, Blue，红绿蓝）三通道，统一缩放为 128×128。

原始像素值	原始含义	训练目标	有效区域
1	宠物前景	1	1
2	背景	0	1
3	未确定区域	占位为 0	0：不计损失与指标

程序用`target` 保存二值目标，用`valid` 标记参与损失和指标计算的位置。原标注为 3 的位置满足`valid=0`，其目标值仅作占位。



上图来自官方训练划分中的样本。左为图片，中为原始标注（橙色宠物、深蓝背景、红色未确定），右为本实验使用的目标；右图白色为宠物、黑色为背景、灰色为忽略区域。图中的空间对齐是监督信号成立的前提。

3.2 划分与变换

保留官方测试集；只在官方 `trainval.txt` 名单内按品种分组，使用固定种子 2026 划分 80% 训练、20% 验证（每类训练数量向下取整）。`splits.json` 保存明确名单，运行记录保存其指纹。生成后为 2941 张训练、739 张验证、3669 张测试。它与控制网络初始化的训练种子 42 用途不同。

图片采用双线性插值缩放，标签采用最近邻插值保留类别编号。训练时的水平翻转同时作用于图片和掩码；验证与测试采用确定的缩放变换。所有模型统一使用 128×128 网格，缩放后原图的长宽比例可能发生变化。

检查点

运行`preview_data.py` 观察图片与掩码。说明前景、背景和忽略区域的对应关系，以及同步变换如何保持像素与标签对齐。

4 T2：卷积、激活与池化怎样执行

4.1 卷积核是可学习的局部计算规则

邱锡鹏《神经网络与深度学习》第 5 章用局部连接和权重共享解释卷积层。本实验沿用该概念，层号使用下标，样本编号使用括号上标。以下单通道数值例子用于说明滑窗计算。

设输入矩阵为 $\mathbf{X} \in \mathbb{R}^{3\times 3}$，卷积核为 $\mathbf{K} \in \mathbb{R}^{2\times 2}$，偏置为 0，步长为 1 且不填充：

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \qquad \mathbf{K} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}. \tag{4.1.1}$$

卷积核在输入上移动，每次对齐窗口内的元素，逐元素相乘再求和。PyTorch 的`Conv2d` 实际执行不翻转核的互相关，课程中仍习惯称其为卷积。左上角输出为：

$$z_{11} = 1\times 1 + 2\times 0 + 4\times 0 + 5\times(-1) = -4. \tag{4.1.2}$$

这里 z_{11} 是激活前得分矩阵 $\mathbf{Z}$ 的第 1 行第 1 列。向右、向下重复同一组权重，得到

$$\mathbf{Z} = \begin{bmatrix} -4 & -4 \\ -4 & -4 \end{bmatrix}. \tag{4.1.3}$$

4.2 把公式变成循环

运行 `python operator_exercises.py` 查看 PyTorch 示例，随后补全 `correlate2d` 函数。本练习实现二维、步长 1、无填充的运算；完整网络使用 PyTorch 基础层及自动求导。

```
window = image[row:row+kh, col:col+kw]
# Hint: multiply corresponding entries, then sum.
```

修改一个值

把 $\mathbf{X}$ 中心的 5 改为 10。计算四个输出，并说明中心像素在各窗口中对应的核权重如何影响结果。

局部连接与权重共享

每个输出位置由局部窗口内的输入计算得到；同一输出通道在不同空间位置使用同一组核权重。窗口与卷积核逐元素相乘后求和，形成该位置的响应。

4.3 激活改变数值，池化改变空间尺寸

ReLU（Rectified Linear Unit，修正线性单元）对每个元素计算 $g(z)=\max(0,z)$，输入和输出形状相同。负数变成 0，正数保留；它为多层网络引入非线性。上一页的 $\mathbf{Z}$ 经 ReLU 后全为 0。这一结果表示该卷积核在当前输入上的响应均为非正值。

最大池化（邱书称最大汇聚）从局部窗口选最大值。对自编激活矩阵 $\mathbf{A}\in\mathbb{R}^{4\times4}$ 使用 2×2 窗口、步长 2：

$$\mathbf{A}=\begin{bmatrix}1&3&2&0\\4&2&1&5\\0&6&3&1\\2&1&4&2\end{bmatrix}\longrightarrow\begin{bmatrix}4&5\\6&4\end{bmatrix}. \tag{4.3.1}$$

每个输入通道独立池化，通道数不变。池化没有可训练的卷积权重；它降低计算量，也舍弃部分位置细节。**补全 T2.2 与 T2.3 后运行：**

```
python checks.py operators
```

4.4 输出尺寸与参数量

高度方向的输入、输出尺寸分别为 H_{in} 和 H_{out}；卷积核大小为 k，两侧填充各 p 个像素，步长为 s。本实验膨胀率固定为 1：

$$H_{\text{out}}=\left\lfloor\frac{H_{\text{in}}+2p-k}{s}\right\rfloor+1. \tag{4.4.1}$$

$\lfloor\cdot\rfloor$ 表示向下取整；宽度方向同理。128 像素输入经过 $k=3,p=1,s=1$ 卷积仍为 128，再经过 $k=2,p=0,s=2$ 池化变为 64。

若输入通道数为 C_{in}、输出通道数为 C_{out}，普通二维卷积的参数量（包含每个输出通道的偏置）为：

$$P=C_{\text{out}}(C_{\text{in}}k^2+1). \tag{4.4.2}$$

例如 3 通道到 16 通道、3×3 卷积有 $16(3\times9+1)=448$ 个参数。空间分辨率不出现在该式中，这体现了权重共享。

检查点

3×3 卷积把 16 通道变为 32 通道，参数量是多少？池化后通道数是否减半？把答案填入报告。

5 T3：自己搭建编码器

5.1 读懂四维张量

张量是多维数组。PyTorch 图像批量采用 `[B,C,H,W]` 顺序：B 为本批图片数，C 为通道数，H,W 为高、宽。样本位于第 0 维。原图的三个通道对应红绿蓝，中间特征的各通道表示网络学习到的不同响应。

`PetEncoder` 继承`nn.Module`。在`__init__` 中创建并注册层，在`forward` 中按顺序调用这些层。训练时，多次前向传播使用同一组可学习参数。

5.2 三阶段固定接口

位置	形状	阶段作用
输入	`[B,3,128,128]`	保留空间结构
`e1`	`[B,16,64,64]`	阶段 1 后空间减半
`e2`	`[B,32,32,32]`	输入通道匹配上一阶段
`z`	`[B,64,16,16]`	最深层特征仍为四维

代码变量`z` 表示编码器的最深层特征；第 7 节的数学记号 $z_u^{(n)}$ 表示最终输出头在某个像素上的激活前得分。

T3.1–T3.3： 在`student_encoder.py` 中用`nn.Sequential` 组织卷积、ReLU、池化。**T3.4：** 依次计算并返回`e1,e2,z`。下列语法中的`in_channels` 和`out_channels` 分别填入各阶段的输入、输出通道数：

```
nn.Conv2d(in_channels, out_channels, kernel_size=3,
          stride=1, padding=1)
nn.ReLU()
nn.MaxPool2d(kernel_size=2, stride=2)
```

```
python checks.py encoder
```

检查程序在本地 CPU 上运行，使用两个程序生成的 `[3,128,128]` 输入张量。程序输出各阶段的尺寸，并检查梯度计算和一次参数更新。将输出结果与逐层尺寸表一同记入报告。

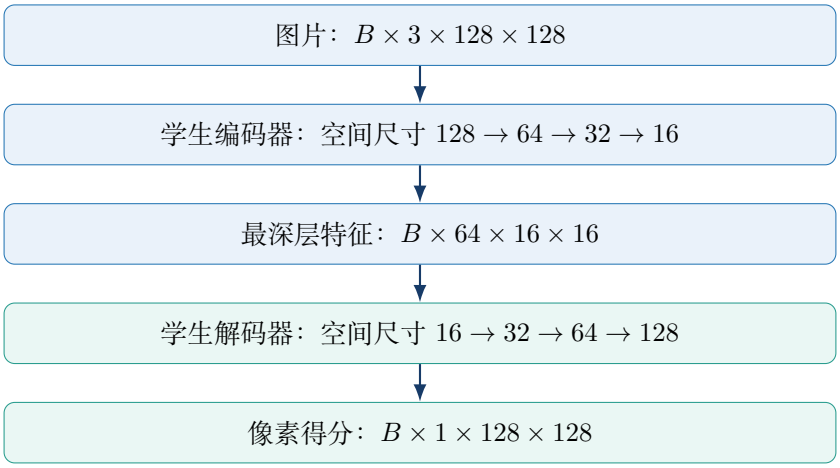
层间连接关系

后一层的输入通道数等于前一层的输出通道数。每个阶段通过一次池化将高、宽各缩小一半，并保留批量、通道与二维空间结构，供解码器继续处理。

6 T4：自己搭建解码器

6.1 从最深层特征恢复逐像素输出

编码器把局部像素变成分辨率更低的特征；解码器把这些特征转成逐像素输出。这里的 backbone 指承担主要特征提取的编码器。修改`student_decoder.py` 中的`PetDecoder`，基础模型只接收最深层特征`z`。



蓝色表示编码路径，青色表示解码路径，箭头表示数据流。两部分均由学生实现，`models.py` 提供完整网络的连接。

6.2 逐步形状与层配置

以下为 128×128 输入下的固定接口。B 表示批量大小；所有 3×3 卷积使用步长 1、填充 1，后接 ReLU。表中的上采样使用双线性插值，保持通道数。

变量或层	输出形状	本步操作
`features`	`[B,64,16,16]`	编码器输出`z`
`up2`	`[B,64,32,32]`	上采样，高、宽各乘 2
`block2/d2`	`[B,32,32,32]`	64 到 32 通道的卷积与激活
`up1`	`[B,32,64,64]`	上采样，高、宽各乘 2
`block1/d1`	`[B,16,64,64]`	32 到 16 通道的卷积与激活
`up0`	`[B,16,128,128]`	上采样至`output_size`
`block0/d0`	`[B,16,128,128]`	16 到 16 通道的卷积与激活
`logits`	`[B,1,128,128]`	`output`：1×1 卷积，16 到 1 通道

最后一层返回原始得分，训练损失在内部处理 sigmoid；概率与二值掩码在预测阶段计算。基础版和改进版共用同一解码器结构，以比较编码器改动的影响。

6.3 需要修改的位置与 Hint

任务	修改位置	Hint
T4.1	`__init__` 中的三个`block`	使用`nn.Sequential` 组织卷积和激活，通道数按上一页填写
T4.2	`__init__` 中的`output`	1×1 卷积用于组合通道，输出通道数为 1
T4.3	`forward` 中的`up2/up1/up0`	前两次指定`scale_factor=2`，最后一次指定`size=output_size`
T4.4	`forward` 中的`d2/d1/d0/logits`	按数据流调用对应层，每一层接收上一步结果

在文件开头已导入`nn` 和`F`。下面展示一次上采样与卷积块调用的写法，其余步骤按形状表完成：

```
x = F.interpolate(features, scale_factor=2,
                  mode="bilinear", align_corners=False)
x = self.block2(x)
```

`__init__` 负责创建可学习层，`forward` 负责调用层。插值扩大空间尺寸，卷积学习特征组合。`output_size` 由`models.py` 从原图高、宽传入。

补全后在 CPU 上分别检查解码器、基础模型和改进模型：

```
python checks.py decoder
python checks.py basic
python checks.py improved
```

解码器检查打印三个卷积块和输出头的尺寸，并计算输入与参数的梯度。完整模型检查连接关系及一次参数更新。把结果与形状表记入报告。

检查点

一次上采样把 `[B,64,16,16]` 变为 `[B,64,32,32]`，为什么通道数保持 64？若希望输出变为 32 通道，应修改哪一步？

参考要点：插值独立作用于每个通道的空间网格；通道数由后续卷积的输出通道参数决定。

7 损失、梯度与分割指标

7.1 从像素标签得到训练目标

对第 n 张图片的第 u 个空间位置，$y_u^{(n)} \in \{0,1\}$ 表示背景或宠物，$m_u^{(n)} \in \{0,1\}$ 表示是否有效。$z_u^{(n)} \in \mathbb{R}$ 是网络输出的原始得分，$p_u^{(n)}$ 是宠物概率。sigmoid 函数把实数映射到 $(0,1)$：

$$p_u^{(n)} = \sigma(z_u^{(n)}) = \frac{1}{1+\exp(-z_u^{(n)})}. \tag{7.1.1}$$

exp 是自然指数函数。把每个有效像素视为一个伯努利标签预测问题，负对数似然给出**二元交叉熵**。网络利用图像中的空间信息预测每个像素，训练目标由各有效位置的损失汇总得到。

$$\mathcal{L}_{\text{pix}}(z,y) = -y\log\sigma(z) - (1-y)\log(1-\sigma(z)). \tag{7.1.2}$$

log 为自然对数；$\mathcal{L}_{\text{pix}}$ 为局部像素损失，整张图和批量损失仍统一使用 $\mathcal{L}$。设 $M^{(n)} = \sum_u m_u^{(n)} > 0$ 为有效像素数，则

$$\mathcal{L}^{(n)} = \frac{\sum_u m_u^{(n)} \mathcal{L}_{\text{pix}}(z_u^{(n)}, y_u^{(n)})}{M^{(n)}}, \qquad \mathcal{L}_B = \frac{1}{B}\sum_{n=1}^{B} \mathcal{L}^{(n)}. \tag{7.1.3}$$

这里 $\sum$ 表示求和。分母按每张图的有效像素数计算，使图片在批量中等权。

7.2 一次可核对的更新

偏导符号 ∂ 描述改变一个变量时函数的变化率。记 $p = \sigma(z)$，则 $\mathrm{d}p/\mathrm{d}z = p(1-p)$。由链式法则得到

$$\frac{\partial \mathcal{L}_{\text{pix}}}{\partial z} = \left(-\frac{y}{p} + \frac{1-y}{1-p}\right) p(1-p) = p - y. \tag{7.2.1}$$

乘上有效掩码后，忽略位置的梯度为 0。若一张教学小图只有两个有效像素，标签为 $[1,0]$，得分为 $[0,0]$，则概率为 $[0.5,0.5]$，平均损失为 $\log 2 \approx 0.6931$。把两个得分写成向量 $\mathbf{z} \in \mathbb{R}^2$，对它的梯度为

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}} = \frac{1}{2}[0.5-1,\ 0.5-0] = [-0.25,\ 0.25]. \tag{7.2.2}$$

把这两个得分暂当可直接更新的变量，学习率 $\eta = 0.1$ 时，一步梯度下降得到 $[0.025, -0.025]$。这一更新使前景得分增大、背景得分减小。完整网络通过链式法则，将得分梯度继续传递到卷积核和偏置等参数。

得分、概率与预测掩码

训练时，`binary_cross_entropy_with_logits` 直接接收模型输出的原始得分，并在内部完成数值稳定的计算。预测时，sigmoid 将得分转为宠物概率，再按阈值 0.5 得到二值掩码。

7.3 用区域重合程度评价分割

如果背景占 90%，全预测背景也有 90% 像素准确率，但没有找到宠物。我们主要评价前景区域的重合程度。在有效区域内，设 P 是预测宠物像素集合，G 是真实宠物像素集合，$|P|$ 表示集合中的像素个数：

$$\mathrm{IoU} = \frac{|P \cap G|}{|P \cup G|}, \qquad \mathrm{Dice} = \frac{2|P \cap G|}{|P| + |G|}. \tag{7.3.1}$$

IoU（Intersection over Union）为交并比；$\cap$ 取共有像素，$\cup$ 取至少属于一个集合的像素。Dice 为另一种重合系数，二者越大越好。预测使用 $p \geq 0.5$，等价于原始得分 $z \geq 0$。

手算评价

预测前景有 3 个像素，真实前景也有 3 个，其中 2 个重合。并集有 $3+3-2=4$ 个，因此 IoU 为 $2/4 = 0.5$，Dice 为 $4/6 \approx 0.6667$。若另有一个忽略位置被预测为前景，它不改变这两个结果。

7.4 统一评价口径

- 评价网格为缩放后的 128×128 标签。
- 先排除原始标签为 3 的位置，再计算每张图的前景 IoU 和 Dice。
- 先获得每张图的分数，再对全部图片取算术平均，每张图权重相同。
- 双方前景都为空时记 1；只有一方为空时记 0。
- 主指标为平均前景 IoU，辅助指标为平均前景 Dice；两者均评价宠物区域。

7.5 训练、验证和测试的职责

训练集用于反向传播；每轮在验证集上计算指标，保存验证 IoU 最高的轮次（并列保留较早轮次）。网络结构和超参数由验证结果确定；测试集用于报告选定模型的最终表现。

8 T5：本地 GPU 与 107 平台两种运行方式

8.1 方案 A：本地 NVIDIA GPU

本地具有 NVIDIA GPU 并配置 CUDA（Compute Unified Device Architecture，统一计算设备架构）版 PyTorch 时，可在本地完成全部训练与评估。安装版本沿用第 1 节；CUDA 安装包按PyTorch 官方历史版本说明选择与设备和驱动匹配的版本。在当前 Python 环境运行：

```
python -c "import torch; print(torch.cuda.is_available())"
```

输出`True`表示当前环境可调用 CUDA 设备。没有本地 NVIDIA GPU 的同学在 CPU 上完成 T2–T4，随后按第 8.2 节使用 107 平台。

已提供`run_local.py`，Windows 与 Linux 使用相同命令。在`code/`中运行基础版与改进版：

```
python run_local.py basic --output outputs/basic-local-01
python run_local.py improved --output outputs/improved-local-01
```

入口先在 CPU 上运行编解码器和完整模型检查，再调用`train.py`在 CUDA 设备上训练。默认 20 轮、批量 16、学习率 0.001、种子 42，与 Slurm 路径一致。可通过`--epochs`、`--batch-size`、`--lr`和`--seed`设置训练预算，各对照组保持一致。

`--output`指定训练输出目录，每次实验使用新的目录名；省略时按模型名和时间生成。`logs/`自动保存`.log`文件，同时显示终端输出，包含命令、模型检查、设备和训练记录。

根据验证集选择模型后运行最终测试。以下示例假定选择改进版，按实际选择替换权重路径：

```
python run_local.py evaluate --checkpoint outputs/improved-local-01/best.pt
```

评估入口调用权重旁`source/evaluate.py`恢复模型，结果写入 outputs/evaluate-local-时间/。本地日志与`run.json`、曲线、代码快照一并提交；本地运行记录中的`slurm_job_id`为空。

8.2 方案 B：107 平台 Slurm

将完整code/ 和数据目录上传至 107 平台，在登录 / 提交终端进入code/。环境解释器使用实验 04 的路径示例：

```
export LAB05_PYTHON="$HOME/.conda/envs/compute-resnet/bin/python"
export LAB05_DATA_DIR="$PWD/data"
export LAB05_EPOCHS=20
export LAB05_BATCH_SIZE=16
export LAB05_LR=0.001
export LAB05_SEED=42
bash scripts/submit.sh basic
bash scripts/submit.sh improved
```

基础版与改进版作业先在 CPU 上执行模型检查，再在 GPU 计算节点训练。检查输出和训练记录写入 Slurm 标准输出日志.out，错误输出写入.err。输出位于 outputs/模型名-作业号/，日志位于logs/。

默认分区为Students，QOS（Quality of Service，服务质量）为qos_stu_default；申请 1 张 GPU、4 个 CPU 核心、8GB 主内存，时限 1 小时。资源参数按账号配置设置，可在提交命令末尾追加sbatch 选项，例如--time=02:00:00。

用 squeue -u "$USER" 查看作业状态，tail -f logs/实际日志名.out 观察进度，scancel 作业号取消作业。根据验证集选择模型后，将对应权重传入测试作业：

```
export LAB05_CHECKPOINT="$PWD/outputs/improved-实际作业号/best.pt"
bash scripts/submit.sh evaluate
```

两条路径使用相同的数据、损失、指标和默认训练预算。评分中的运行记录可采用本地.log 或 Slurm.out/.err，报告注明所用设备与运行方式。

8.3 训练程序的关键顺序

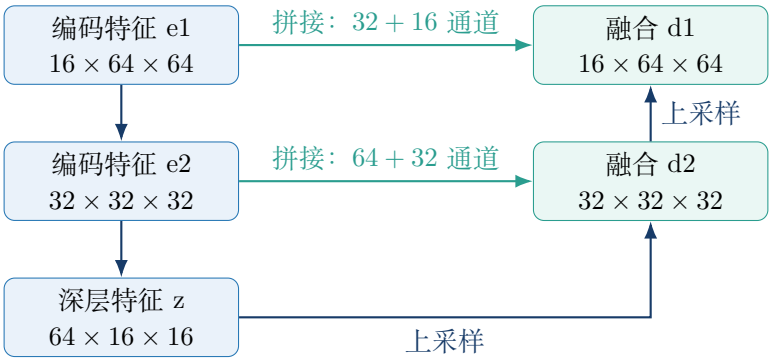
```
model.train()
optimizer.zero_grad()
logits = model(images)
loss = masked_loss(logits, targets, valid)
loss.backward()
optimizer.step()
```

验证时切换到eval() 和torch.no_grad()，根据验证结果选择模型。记录run.json、history.csv 和曲线，用于分析损失与分割指标的变化。

9 进阶选做：自行实现小型 U-Net

9.1 把浅层细节送回解码器

基础解码器只收到 16×16 特征。U-Net 增加编码器到解码器的跨层连接，让解码器同时看到深层表示和相对精细的空间特征。本实验采用两条跳跃连接的教学简化结构，分别融合 32×32 和 64×64 尺度的编码特征。



图中省略批量维，青色横箭头为跳跃连接，深色箭头为顺序路径。d1 再上采样至 128×128，自行设计输出头产生单通道得分。本节为独立选做，学生根据结构图自行编写 U-Net，并接入已有的训练与评估流程。

9.2 实现步骤与维度提示

步骤	自行实现	Hint
1	第一次融合卷积	拼接后 96 通道，卷积输出 32 通道
2	第二次融合卷积	拼接后 48 通道，卷积输出 16 通道
3	第一处融合	`torch.cat([up2,e2],dim=1)` 后调用融合块
4	第二处融合	空间对齐后拼接`up1` 与`e1`

`F.interpolate(...,size=skip.shape[-2:])` 明确对齐高和宽；`dim=1` 沿通道拼接。保持最终输出为 `[B,1,128,128]`，继续使用同一损失与指标。另建模型文件，并扩展`models.py` 的`build_model`、`train.py` 的模型选项、`checks.py` 的模型检查入口和所选运行入口的训练分支。记录各融合阶段的形状和参数量，沿用基础实验的数据划分与训练预算，完成训练、验证和结果分析。

10 联系 ResNet：跳跃连接怎样组合特征

10.1 相加与拼接的共同点和区别

两者都让信息通过跨层路径传播，但组合方式不同。残差相加为优化提供较直接的路径；U-Net 拼接保留两组通道，供后续卷积学习如何融合。

设输入特征张量 $\mathbf{X} \in \mathbb{R}^{B\times C\times H\times W}$，$F$ 是若干层组成的可学习变换，输出形状与输入相同。残差块的相加为

$$\mathbf{Y} = F(\mathbf{X}) + \mathbf{X}. \tag{10.1.1}$$

它对相同形状的张量逐元素相加，输出形状保持不变。若通道或步长不同，可对捷径使用 1×1 投影卷积，使其与主支路匹配。

U-Net 中设上采样特征 $\mathbf{U} \in \mathbb{R}^{B\times C_u\times H\times W}$、跳跃特征 $\mathbf{E} \in \mathbb{R}^{B\times C_e\times H\times W}$。沿通道拼接得到

$$\mathbf{C} = \mathrm{Concat}_{\mathrm{channel}}(\mathbf{U}, \mathbf{E}) \in \mathbb{R}^{B\times (C_u+C_e)\times H\times W}. \tag{10.1.2}$$

Concat 表示连接数组；输入通道可以不同，但批量、高、宽应相同。随后卷积的`in_channels` 为 $C_u + C_e$，即两支通道数之和。

比较项	残差相加	U-Net 通道拼接
代码形式	`main+shortcut`	`torch.cat(...,dim=1)`
输出通道	保持匹配后的通道数	两支通道数之和
关注点	学习输入的修正量	融合语义与定位细节

10.2 连接方式与信息传递

残差相加在匹配的特征空间中叠加输入与学习到的修正；通道拼接将两组表示并列保留，再由后续卷积完成融合。前者关注深层网络的优化路径，后者在本实验中用于结合不同尺度的语义信息与空间细节。

检查点

两个张量都是 `[B,32,32,32]`，相加与拼接分别得到什么形状？如果一支为 64 通道，哪种操作可直接进行？

11 改进提示：归一化与泛化

11.1 批量归一化的使用位置

BN（Batch Normalization，批量归一化）通过归一化特征的均值和方差，调节中间特征的数值尺度，有助于稳定训练中的梯度传播。对 `[B,C,H,W]` 特征，`BatchNorm2d(C)` 在训练时对每个通道汇总批量和空间位置的统计量，随后使用可学习的缩放和平移。

```
nn.Conv2d(16, 32, 3, padding=1)
nn.BatchNorm2d(32)
nn.ReLU()
```

以上是按顺序放入`nn.Sequential` 的层。默认设置下，`train()` 使用当前批次统计量并更新运行统计；`eval()` 使用运行统计。批量很小、样本相关性高或训练测试分布不同，都可能影响效果。每通道还包含缩放 γ 和平移 β 两个可训练参数，参数统计要计入它们。

11.2 层归一化的作用与统计范围

LN（Layer Normalization，层归一化）对每个样本指定的最后若干维进行归一化，以调节特征尺度。统计量在单个样本内计算，训练和评估均使用当前输入，适合批量大小变化或较小批量的场景。PyTorch 的`normalized_shape` 决定归一化范围。

- `LayerNorm([C,H,W])`：对每张图该层的全部通道与空间位置归一化；该设置对应固定的特征形状，默认包含 $C \times H \times W$ 个缩放参数和同样多的平移参数。
- `LayerNorm(C)`：只处理最后一维。如果希望对每个像素的通道向量归一化，先把通道移到最后，再移回。报告中记录所选统计范围与参数量。

```
# In __init__: self.norm = nn.LayerNorm(channels)
# In forward, x has shape [B,C,H,W]:
x = x.permute(0, 2, 3, 1)    # [B,H,W,C]
x = self.norm(x)
x = x.permute(0, 3, 1, 2)    # [B,C,H,W]
```

归一化与泛化

归一化调节网络内部特征的尺度，并通过可学习参数保留表达能力。泛化表现由验证集衡量；实验中比较相同训练预算下的验证指标、损失曲线与预测样例。

11.3 控制变量的对照实验

先保留基础版，再在`ImprovedEncoder` 中只改一个主要因素。固定划分、输入分辨率、训练种子、轮数、批量大小与学习率，比较验证 IoU、损失曲线和参数量。若加入数据增强，应作为另一组独立实验。训练损失反映模型对训练数据的拟合，验证指标反映模型对未参与参数更新的数据的表现。

12 T6：对比、评分与最终提交

12.1 至少两组实验

基础要求为一个基线和一个改进编码器，使用本人实现的同一解码器结构。进阶选做 U-Net，由学生自行编写。默认训练预算为 20 轮、批量 16、学习率 0.001；各对照组使用相同预算，配置与实际耗时记录在报告中。

在验证样例中选至少 3 个成功和 3 个失败案例，分析耳朵、腿、毛发边缘、相似背景和小目标上的预测表现。报告记录各组指标，并结合曲线和预测掩码解释性能变化。评估器保存`best_worst_examples.png`，前三行是 IoU 最低、后三行是最高的样例，展示模型在不同图像上的表现差异。

12.2 评分办法

分值	项目	评分依据
20	基础运算与尺寸解释	手算过程 8，数据和标注解释 4，尺寸与参数表 8
30	编解码器与连接	编码器 10，解码器 10，连接与输出 5，梯度更新检查 5
20	训练与复现	有效运行日志 8，配置/种子/划分记录 6，权重与代码快照 6
20	对比与错误分析	控制变量 8，曲线与选择依据 6，样例分析 6
10	最终分割效果	完整真实测试集上平均前景 IoU 乘 10

效果分使用 0–1 的小数值，例如 IoU 为 0.65 则得 6.5 分，保留一位小数。进阶最多加 20 分：连接和维度正确 8 分、有效对照 8 分、与残差思想的理论比较 4 分；总分封顶 100 分。

12.3 最终测试与提交文件

选定模型后，按第 8.1 节的本地入口或第 8.2 节的 Slurm 入口运行一次最终测试。评估脚本使用对应训练输出的`source/` 代码快照与`best.pt` 权重恢复模型。提交以下内容：

1. 学生代码与报告，报告包括基础运算结果、逐层尺寸表、对比表和结果图片。
2. 基础与改进训练的`run.json`、`history.csv`、曲线及运行日志；日志中包含模型尺寸与参数更新的检查结果。
3. 最终选定模型的`best.pt` 及对应`source/` 代码快照。
4. 最终测试的`metrics.json`、`per_image.csv` 和`examples.png`。

最终压缩包命名为学号 _ 姓名 _lab05.zip，报告导出为`report.pdf`。完整目录、复现命令和打包检查见同目录《提交与打包说明.md》，报告从`report-template.md` 开始填写。

提交包包含本人代码、报告与实验记录，数据来源和环境配置写入复现说明。收件渠道与截止时间按课程通知执行；训练与评估可在本地 NVIDIA GPU 或 107 平台完成。

13 复习与符号索引

13.1 退出卡与课后复核

1. 池化保留什么、丢失什么？上采样能否完整恢复原信息？
2. `[B,64,32,32]` 与 `[B,32,32,32]` 拼接后有多少通道？
3. 原始得分、sigmoid 概率与二值预测掩码如何对应？训练损失接收哪一种量？
4. BN 和 LN 的统计范围分别是什么？如何用对照实验评价它们的作用？

参考要点：池化保留局部响应摘要并丢失部分位置细节；插值不能唯一找回原信息。拼接为 96 通道。训练损失接收原始得分并内部稳定处理；预测概率由 sigmoid 得到。BN 按通道汇总批量和空间位置，LN 按单个样本的指定维度计算；其效果通过相同划分和预算下的验证结果比较。

13.2 符号索引

类别	记号	含义与首次说明
数据与模型	B, C, H, W	批量、通道、高、宽；第 5.1 节
数据与模型	$y_u^{(n)}, m_u^{(n)}$	像素标签、有效掩码；第 7.1 节
概率与目标	$z_u^{(n)}, p_u^{(n)}$	原始得分、宠物概率；第 7.1 节
概率与目标	$\mathcal{L}^{(n)}, \mathcal{L}_B$	单图损失、批量平均损失；第 7.1 节
函数与算子	$\sigma, \exp, \log$	sigmoid、指数、自然对数；第 7.1 节
函数与算子	$\lfloor \cdot \rfloor$	向下取整；第 4.4 节
集合与评价	$P, G, \lvert\cdot\rvert, \cap, \cup$	前景集合、个数、交并运算；第 7.3 节

14 参考资料与学习接口

14.1 实际使用的本地资料

本实验配合第 5 次理论课的卷积网络、残差连接与归一化内容。以下路径相对于课程仓库。

- 教案/人工智能数学原理与算法 _12 次课程与助教分工 _ 最终版.xlsx：“课程执行总表”“教学深度与时间”，对齐 10 月 19 日卷积、残差与 BN 内容。
- `source/pdf/nndl-v2.pdf`：邱锡鹏《神经网络与深度学习》，第 5.1–5.2 节卷积与汇聚、第 5.4.5 节残差网络；相关内容见书中第 125、137 页（文件第 138、150 页）。
- 教案/第五次教学 _ 卷积神经网络与残差网络/第五次教学 _ 卷积神经网络与残差网络.tex：通道、尺寸、参数量和残差连接的课程记号与知识边界。
- 实验 04 讲义及`code/compute-practice/scripts/`：107 平台的提交终端、资源参数、环境解释器与日志工作流。

14.2 数据、模型与接口资料

- Oxford-IIIT Pet 官方数据页及下载包`annotations/README`：宠物图片、三值掩码与官方划分。
- Ronneberger 等，U-Net（2015）：编码路径、解码路径和拼接连接。本实验采用两条跳跃连接的教学简化版。
- He 等，Deep Residual Learning（2015）：残差学习与快捷连接。
- PyTorch 官方文档：OxfordIIITPet、BatchNorm2d、LayerNorm：数据集接口、归一化维度与参数说明；依赖版本见`requirements.txt`。

14.3 把实验放回完整工作流

任务与代价是“找出主体，兼顾漏分与误分”；数据阶段核对标注并固定划分；表示阶段由学生编码器学习特征；模型阶段连接解码器；目标阶段使用忽略边界的交叉熵；优化阶段执行梯度更新；选择阶段看验证集；最后测试并分析错误。在真实产品中，还需检查不同拍摄条件、多主体图像、时延及上线后的数据变化。

下一步学习时，可以继续追问：不同尺度怎样融合？残差和归一化如何迁移到更深的网络？本实验保存的尺寸表、控制变量方法和代码快照可直接用于后续网络实验。